

ui.radio 全面详解（基于 NiceGUI 文档）

ui.radio 是 NiceGUI 中用于实现多选项互斥选择的核心组件，基于 Quasar 的 QOptionGroup 组件实现。它支持列表或字典形式配置选项，具备数据绑定、状态监听、自定义选项内容等能力，适用于表单选择、功能模式切换等需要“二选一”或“多选一”的场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QOptionGroup 组件，继承其互斥选择逻辑、样式体系和无障碍支持。
- 核心功能：支持以列表（值即标签）或字典（键为值、值为标签）配置选项，默认互斥选择，支持动态修改选项和选中值。
- 灵活扩展性：通过 `ui.teleport` 可向选项中注入任意内容（如图标、图片），突破默认文本标签限制。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
options	list / dict	选项配置，列表格式为 <code>[值1, 值2, ...]</code> （值即标签），字典格式为 <code>{值1: 标签1, 值2: 标签2, ...}</code>	-
value	任意类型	初始选中值（需与选项中的值类型一致）	-
on_change	Callable	选中值变化时触发的回调函数（ <code>e.value</code> 为当前选中值）	-

二、基础使用示例

1. 两种选项配置方式（列表 / 字典）

展示列表格式（值即标签）和字典格式（值与标签分离）的基础用法，支持横向排列：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 列表格式选项（值即标签，横向排列）
    ui.label('列表格式（值=标签）：')
    radio1 = ui.radio([1, 2, 3], value=1).props('inline') # inline 实现横向排列

    # 2. 字典格式选项（值与标签分离，横向排列）
    ui.label('字典格式（值≠标签）：')
    radio2 = ui.radio({1: '选项A', 2: '选项B', 3: '选项C'}).props('inline')

    # 3. 绑定两个 radio，选中值同步
    radio2.bind_value(radio1, 'value')

    # 显示当前选中值
```

```
ui.label().bind_text_from(radio1, 'value', lambda x: f'当前选中值: {x}')

ui.run()
```

效果:

- 第一个 radio 以数字为值和标签, 默认选中 1;
- 第二个 radio 以数字为值、文本为标签, 通过 `bind_value` 与第一个 radio 同步选中值;
- 两个 radio 均通过 `props('inline')` 实现横向排列, 默认纵向排列。

2. 纵向排列与状态监听

默认纵向排列, 结合 `on_change` 监听选中值变化, 适用于选项较多的场景:

```
from nicegui import ui

# 纵向排列的 radio 组, 监听选中变化
ui.radio(
    options=['Python', 'JavaScript', 'Java', 'C++'],
    value='Python',
    on_change=lambda e: ui.notify(f'选中编程语言: {e.value}')
)

ui.run()
```

效果: 四个选项纵向排列, 初始选中 Python, 点击其他选项触发通知, 显示当前选中值。

三、核心功能与进阶用法

1. 动态修改选项与选中值

通过 `set_options` 动态更新选项列表, 通过 `set_value` 手动设置选中值, 适用于动态加载选项的场景:

```
from nicegui import ui

# 初始选项
radio = ui.radio(['基础版', '进阶版'], value='基础版')

# 动态添加选项
def add_option():
    new_options = radio.options + ['专业版'] # 新增选项
    radio.set_options(new_options)
    radio.update() # 刷新界面

# 手动设置选中值
def set_to_pro():
    radio.set_value('专业版')

# 控制按钮
with ui.row().classes('mt-4 gap-2'):
    ui.button('添加专业版选项', on_click=add_option)
```

```

        ui.button('选中专业版', on_click=set_to_pro)

# 显示当前选中值
ui.label().bind_text_from(radio, 'value', lambda x: f'当前版本: {x}')

ui.run()

```

效果：初始显示两个选项，点击“添加专业版选项”后新增第三个选项，点击“选中专业版”可手动切换选中值，下方标签实时同步。

2. 注入任意内容到选项（图标 / 图片）

通过 `ui.teleport` 向选项标签中注入图标、图片等内容，突破默认文本标签限制：

```

from nicegui import ui

# 配置空标签的选项（后续通过 teleport 注入内容）
options = ['Star', 'Thump Up', 'Heart']
radio = ui.radio({x: '' for x in options}, value='Star').props('inline')

# 向每个选项的标签中注入图标
with ui.teleport(f'#{radio.html_id} > div:nth-child(1) .q-radio__label'):
    ui.icon('star', size='md', color='yellow')
with ui.teleport(f'#{radio.html_id} > div:nth-child(2) .q-radio__label'):
    ui.icon('thumb_up', size='md', color='blue')
with ui.teleport(f'#{radio.html_id} > div:nth-child(3) .q-radio__label'):
    ui.icon('favorite', size='md', color='red')

# 显示当前选中值
ui.label().bind_text_from(radio, 'value', lambda x: f'选中图标: {x}')

ui.run()

```

效果：三个选项横向排列，标签为图标而非文本，点击图标切换选中状态，下方标签显示选中的图标名称。

3. 数据绑定（双向同步）

通过 `bind_value` 实现 radio 与数据对象的双向绑定，数据变化时组件自动更新，反之亦然：

```

from nicegui import ui

class AppState:
    def __init__(self):
        self.selected_theme = 'light' # 初始选中值

state = AppState()

# radio 与数据对象绑定
radio = ui.radio(
    options={'light': '亮色模式', 'dark': '暗色模式', 'system': '系统模式'},
).bind_value(state, 'selected_theme')

```

```
# 显示绑定数据的实时状态
ui.label().bind_text_from(
    state, 'selected_theme',
    lambda x: f'当前主题: {x}' (数据对象同步更新) '
)

# 手动修改数据对象, 组件自动同步
ui.button('切换为暗色模式', on_click=lambda: setattr(state, 'selected_theme',
'dark')).classes('mt-2')

ui.run()
```

效果: 点击 radio 切换主题, 数据对象 `state.selected_theme` 自动同步; 点击按钮修改数据对象, radio 选中状态自动更新。

4. 样式定制 (排列方式、颜色、尺寸)

通过 `props` 和 `classes` 定制 radio 的排列方式、选中颜色、尺寸等样式:

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 纵向排列+自定义选中颜色
    ui.label('纵向排列+红色选中: ')
    ui.radio(
        ['选项1', '选项2', '选项3'],
        value='选项1'
    ).props('color=red') # 选中颜色为红色

    # 2. 横向排列+紧凑模式+大尺寸
    ui.label('横向排列+紧凑+大尺寸: ')
    ui.radio(
        ['A', 'B', 'C'],
        value='A'
    ).props('inline dense size=lg') # inline=横向, dense=紧凑, size=lg=大尺寸

    # 3. 自定义选项间距 (通过 classes)
    ui.label('自定义选项间距: ')
    ui.radio(
        ['苹果', '香蕉', '橙子'],
        value='苹果'
    ).classes('gap-6') # 选项间距为 6

ui.run()
```

效果: 三个 radio 组分别展示不同样式, 支持颜色、排列方式、尺寸、间距的定制。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
enabled	BindableProperty	是否启用（可绑定数据动态控制，禁用后不可交互）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持，用于 teleport 定位）
options	list / dict	当前选项配置（可动态修改）
value	BindableProperty	当前选中值（可绑定数据动态修改）
visible	BindableProperty	是否可见（可绑定数据）
props	Props[Self]	Quasar 特性属性（如 <code>inline</code> 横向排列、 <code>color</code> 选中颜色）

2. 关键方法

(1) 状态控制

- `disable()`：禁用组件（不可交互，视觉灰度）
- `enable()`：启用组件
- `set_enabled(value: bool)`：设置启用状态（True/False）
- `set_visibility(visible: bool)`：设置组件可见性

(2) 选项与值修改

- `set_options(options: list / dict, value: Any = Ellipsis)`：动态更新选项，可选参数 `value` 用于设置新选中值（未指定则保留当前值）
- `set_value(value: Any)`：动态设置选中值（需与选项中的值类型一致，触发 `on_change` 事件）
- `update()`：修改选项或属性后，调用此方法刷新界面

(3) 事件与绑定

- `on_change(callback)`：绑定选中值变化事件（`e.value` 为当前选中值）
- `on(type: str, handler)`：订阅任意 DOM 事件（如 `click`、`mousedown`）
- `bind_value(target_object, target_name)`：双向绑定选中值到目标对象的属性
- `bind_enabled_from(target_object, target_name)`：单向绑定启用状态从目标对象

(4) 其他实用方法

- `tooltip(text: str)`：为组件添加悬停提示
- `delete()`：彻底删除组件
- `mark(*markers)`：添加标记（用于测试或元素查询）

- `clear()`：移除组件内的所有子元素

五、使用场景与注意事项

1. 适用场景

- 表单选择：如用户性别、学历、偏好等互斥选项的录入。
- 模式切换：如界面主题、数据排序方式、功能模式等的切换。
- 筛选条件：如列表数据的分类筛选（如按状态、类型筛选）。

2. 关键注意事项

- 选项格式一致性：列表格式中选项值即标签，字典格式中键为值、值为标签，需确保 `value` 与选项中的值类型一致（如数字、字符串）。
- 横向排列配置：默认纵向排列，需通过 `props('inline')` 实现横向排列，适用于选项较少的场景。
- 动态选项更新：修改 `options` 属性后，必须调用 `update()` 方法才能刷新界面显示。
- 版本兼容性：`html_id` 属性仅在 2.16.0+ 版本支持，`bind_enabled` 的 `strict` 参数在 3.0.0+ 版本支持，使用时需确认版本。
- 与 `ui.toggle` 的区别：`ui.radio` 更适合表单场景，默认支持多选项集中配置；`ui.toggle` 更适合分散的选项切换，支持单独定制每个选项的图标、颜色。

六、进阶示例：带图片的选项 radio

结合 `ui.teleport` 实现带图片的 radio 选项，适用于商品选择、头像选择等场景：

```
from nicegui import ui

# 选项配置（值为图片ID，标签为空）
image_ids = ['377', '370', '380'] # picsum.photos 的图片ID
options = {img_id: '' for img_id in image_ids}
radio = ui.radio(options, value='377').props('inline')

# 向每个选项注入图片
for idx, img_id in enumerate(image_ids, 1):
    with ui.teleport(f'#{radio.html_id} > div:nth-child({idx}) .q-radio__label'):
        ui.image(f'https://picsum.photos/id/{img_id}/80/80').classes('rounded-md')

# 显示当前选中的图片ID
ui.label().bind_text_from(radio, 'value', lambda x: f'选中图片ID: {x}').classes('mt-2')

ui.run()
```

效果：三个选项横向排列，每个选项显示一张图片，点击图片切换选中状态，下方标签显示当前选中的图片 ID。